

Rappresentazione dei numeri interi

Il problema

Con il sistema binario possiamo rappresentare i numeri positivi; se vogliamo i negativi, dobbiamo anteporre il segno -. Ma un computer non può mettere il segno meno: quindi bisogna trovare un modo per “includere” il segno nel numero. Vedremo quattro rappresentazioni: modulo e segno, complemento a 1 e complemento a 2 e rappresentazione per eccesso.

D’ora in poi quando si parla di codifica si intende dal sistema binario alla rappresentazione in oggetto; decodifica invece significa convertire dal sistema di rappresentazione al sistema decimale.

Modulo e segno (MS)

Nella rappresentazione modulo e segno viene aggiunto un bit a sinistra (diventa l’MSB) alla classica rappresentazione in sistema binario dei numeri: questo vale 0 se il numero è positivo, mentre 1 se è negativo.

Rappresentazione MS

Codifica:

- 1) Si converte il modulo del numero normalmente;
- 2) Si aggiunge un 1 a sinistra se è negativo, 0 se è positivo.

Decodifica:

- 1) Se il primo bit è 1, mettere un -;
- 2) Eliminare il primo bit e convertire il resto del numero normalmente.

Esempio 1

Rappresentiamo in MS alcuni numeri:

$$-7_{10} = \underbrace{1}_{-} \underbrace{111}_7 = 1111_{\text{MS}}$$

$$7_{10} = \underbrace{0}_{+} \underbrace{111}_7 = 0111_{\text{MS}}$$

Convertiamo da MS:

$$111_{\text{MS}} = \underbrace{1}_{-} \underbrace{11}_3 = -3_{10}$$

Il problema di questa rappresentazione è (oltre al fatto che un bit viene “sprecato” per il segno) che lo 0 può essere rappresentato in due modi: sia come -0 che come +0, $0000_{\text{MS}} = 1000_{\text{MS}}$ (ovviamente qui usiamo la rappresentazione MS: altrimenti sarebbero 0 e 8 in sistema binario, conveniamo che siano diversi!).

Qual è il range di numeri rappresentabili con n bit? Uno va usato per il segno, gli altri per il numero. Quindi abbiamo $n - 1$ bit che corrispondono a 2^{n-1} informazioni, che possono essere sia in negativo che in positivo. Per il problema dello 0, tuttavia (che viene considerato sia negativo che positivo) dobbiamo togliere 1 a questa quantità. Quindi possiamo rappresentare i numeri che in base 10 stanno nell'intervallo $[-(2^{n-1} - 1), 2^{n-1} - 1]_{10}$.

Rappresentazioni con complemento

Il complemento

Per parlare del complemento a 1 o a 2 dobbiamo prima capire cos'è il complemento di un numero.

Complemento

Il complemento di un numero a si indica con $\overline{a}$.

Per complementare un numero binario invertiamo il valore di tutti i suoi bit:

$$\overline{b_i} = 1 - b_i.$$

Esempio 1

Complementiamo alcuni numeri binari:

$$\overline{10101_2} = 01010_2$$

$$\overline{1111}_2 = 0000_2$$

$$\overline{11101001}_2 = 00010110_2$$

Complemento a 1 (CA1)

Con l'operazione di complemento che abbiamo appena visto possiamo definire il complemento a 1.

Rappresentazione CA1

Codifica:

Numero positivo: si converte normalmente da sistema decimale a binario e si aggiunge uno 0 a sinistra.

Numero negativo: si converte normalmente il modulo del numero aggiungendo uno 0 a sinistra, poi si fa il complemento.

Decodifica:

MSB = 1: si complementa il numero, poi si converte in decimale.

MSB = 0: si converte in base decimale.

Esempio 1

Rappresentiamo in CA1 alcuni numeri:

$$6_{10} = 0110_{CA1}$$

$$-6_{10} = \overline{0110}_2 = 1001_{CA1}$$

C'è lo stesso identico problema di prima: lo 0 può essere rappresentato sia con tutti 0 che con tutti 1, $0000_{CA1} = 1111_{CA1}$; anche l'intervallo di numeri rappresentabili è lo stesso: $[-(2^{n-1} - 1), 2^{n-1} - 1]_{10}$. Quindi neanche questa va bene, ma è il primo passo per il complemento a 2, che è la “soluzione finale”.

N.B.: Se il numero ha come prima cifra uno 0, è positivo; se la prima cifra è 1 è negativo! Questo vale anche per il CA2.

N.B.: Con la rappresentazione CA1 è fondamentale capire perché va aggiunto lo 0 a sinistra: supponiamo di voler convertire -7.

$$7_{10} = 111_2$$

$$-7_{10} = \overline{111}_2 = 000_{CA1}$$

Questo risultato è assurdo! Infatti quel numero è 0. In verità non abbiamo considerato che se abbiamo 3 bit, la rappresentazione CA1 può rappresentare i numeri da $-(2^{3-1} - 1)$ fino a $2^{3-1} - 1$, quindi da -3 a +3 (infatti, con 3 bit e la CA1 possiamo rappresentare 8 numeri: -3, -2, -1, -0, +0, +1, +2, +3. Che spreco lo 0 così, no?). Quindi il -7 non possiamo esprimerlo. **Se nel sistema binario ci servono n bit per rappresentare un numero, con MS e CA1 ce ne serviranno n+1.** Quindi

$$-7_{10} = \overline{0111}_2 = 1000_{CA1}$$

Ora ha molto più senso! Quando convertiamo i numeri, aggiungiamo sempre uno 0 a sinistra, così passiamo a n+1 bit. Poi se è negativo complementiamo, altrimenti abbiamo finito.

Complemento a 2 (CA2)

Finalmente, la soluzione! Il complemento a 2 risolve i problemi relativi al CA1.

Rappresentazione CA2

Codifica (metodo 1, complemento alla base):

Numero positivo: si converte normalmente da sistema decimale a binario e si aggiunge uno 0 a sinistra.

Numero negativo X : avendo n bit, si converte il numero $2^n - X$.

Codifica (metodo 2, integrazione con CA1):

Numero positivo: si converte normalmente da sistema decimale a binario e si aggiunge uno 0 a sinistra (come CA1).

Numero negativo: si converte normalmente il modulo del numero aggiungendo uno 0 a sinistra, poi si fa il complemento e si aggiunge 1 (ossia si fa CA1 e poi +1).

Codifica (metodo 3, regola pratica):

- 1) Si converte il numero in binario aggiungendo uno 0 a sinistra;
- 2) Se è negativo, si parte da destra, andando verso sinistra si trascrivono tutti i numeri finché non si incontra un 1, da trascrivere anch'esso;

3) Si complementano i bit restanti.

Decodifica (metodo 1, integrazione con CA1):

MSB = 1: si sottrae 1, si complementa, poi si converte in decimale.

MSB = 0: si converte in base decimale.

Decodifica (metodo 2, regola pratica):

ATTENZIONE! Questo metodo non è stato descritto dai prof. Ho giocato un po' coi numeri e sembra che funzioni. Il ragionamento che c'è dietro dovrebbe essere valido, ma non vorrei aver tratto conclusioni sbagliate!)

MSB = 1: si complementano tutte le cifre tranne l'ultima (LSB); si converte in decimale e si mette il meno davanti.

MSB = 0: si converte in base decimale.

Esempio 1

Rappresentiamo in CA2 alcuni numeri:

$$6_{10} = 0110_{CA2}$$

$$-6_{10} = \overline{0110}_2 + 1 = 1001_{CA1} + 1 = 1010_{CA2}$$

E finalmente lo 0 può essere rappresentato solo come 0! L'intervallo di numeri che viene rappresentato con il CA2 infatti è asimmetrico, lo 0 viene considerato come positivo (provate a farlo negativo, se vi va...*hello overflow my old friend*): $[-2^{n-1}, 2^{n-1} - 1]_{10}$.

N.B.: distinguiamo tra l'operazione di complemento a 2 e la rappresentazione in complemento a 2: l'operazione è $2^n - X$, mentre la rappresentazione è quanto descritto sopra. La rappresentazione CA2 sfrutta l'operazione CA2 per convertire i numeri.

Proprietà dell'operazione CA2:

$$2^n - (2^n - X) = X$$

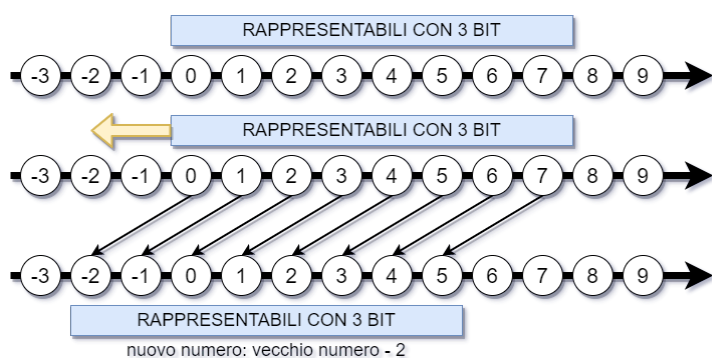
$$CA2(CA2(X)) = X$$

Inoltre, facendo il CA2 di un numero negativo si ottiene il corrispondente positivo.

N.B.: se vogliamo estendere a m bit un numero CA2 a n bit (naturalmente $m > n$) si replica l'MSB tante volte quante serve (esempio, da nybble a byte):
 $0101_{CA2} \rightarrow 00000101_{CA2}$, $1000_{CA2} \rightarrow 11111000_{CA2}$

Rappresentazione in eccesso

La rappresentazione in eccesso n sfrutta un concetto matematico semplicissimo. Mettiamo di avere 3 bit: quindi possiamo rappresentare, con il sistema binario, i numeri da 0 a 7 (che sono 8 numeri). Ma io voglio rappresentare 8 numeri da -2 a 5! Che faccio? Semplicemente traslo il mio intervallo:



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/03/circuiti-logici.png?ssl=1> >

Traslo i numeri di -2: in questo modo posso rappresentare le informazioni che mi servono.

Quindi si crea una funzione biunivoca:

- 1) In codifica, il mio -2 è 0, -1 è 1, 0 è 2...quindi $n + 2$;
- 2) In decodifica, lo 0 è il mio -2, l'1 il mio -1...quindi $n - 2$.

Quindi se il binario vale 1, io so che corrisponde al mio -1 (1-2). Se ho 6, so che il corrispettivo valore è 4 (6-2). Quando codifico faccio l'operazione inversa: il mio 3 corrisponde al "normale" 5 (3+2), quindi la codifica è 5: quando poi otterrò 5 saprò che dovrò togliere 2, e riottengo 3. Ovviamente ora ho fatto tutti i calcoli in base 10, ma poi si codifica il numero in binario dopo aver aggiunto (codifica)/tolto (decodifica) il numero per cui ho traslato.

Rappresentazione in eccesso n

Codifica:

Si converte in binario (numero + n).

Decodifica:

Si converte in base decimale il numero in binario, poi si sottrae n .

Esempio 1

Convertiamo 23 in eccesso 127:

$$23_{10} \rightarrow (23 + 127 = 150 = 10010110_2) \rightarrow 23_{10} = 10010110_{e127}$$

Abbiamo principalmente due rappresentazioni in eccesso: eccesso 127 (utile per la codifica dei numeri reali) ed eccesso 128.